

Algoritmos Paralelos - Memory Consistency Models

8-Abril-2026

These exercises aim to promote a better understanding of OpenMP memory consistency model. (Note: programs should be compiled with optimization enabled)

Complete the following OpenMP programs with “flush” directives for correct behavior.

```
#include <omp.h>
#include <stdio.h>
int main() {
    int data;
    int flag=0;
    #pragma omp parallel
    {
        if (omp_get_thread_num()==0) {
            /* Write to the data buffer that will be read by thread */
            data = 42;
            /* Set flag to release thread 1 */
            flag = 1;
        }
        else if(omp_get_thread_num()==1)
        {
            /* Loop until we see the update to the flag */
            while (flag < 1)
            {
            }
            printf("flag=%d data=%d\n", flag, data);
        }
    }
}
```

```
#include <omp.h>
#include <stdio.h>
int main() {
    int flag=0;
    #pragma omp parallel num_threads(3)
    {
        if(omp_get_thread_num()==0)
        {
            /* Set flag to release thread 1 */
            #pragma omp atomic
            flag++;
        }
        else if(omp_get_thread_num()==1)
        {
            /* Loop until we see that flag reaches 1 */
            while(flag < 1)
            {
            }
            printf("Thread 1 awoken\n");
            /* Set flag to release thread 2 */
            #pragma omp atomic
            flag++;
        }
        else if(omp_get_thread_num()==2)
        {
            /* Loop until we see that flag reaches 2 */
            while(flag < 2)
            {
            }
            printf("Thread 2 awoken\n");
        }
    }
}
```

GPT5 question (please comment the result)

write a simple OpenMP barrier with atomics

Here's a minimal example of implementing a barrier in OpenMP using atomics, without relying on OpenMP's built-in `#pragma omp barrier`. The idea is that each thread signals its arrival and then waits until all have arrived.

```
#include <stdio.h>
#include <omp.h>
int main() {
    int nthreads;
    int count = 0;          // shared counter for arrived threads
    int sense = 0;          // shared sense variable
    #pragma omp parallel shared(count, sense, nthreads)
    {
        int local_sense = 1;
        #pragma omp single
        nthreads = omp_get_num_threads();
        // Each thread does something before the barrier
        int tid = omp_get_thread_num();
        printf("Thread %d reached the barrier\n", tid);
        // Simple barrier using atomics
        #pragma omp atomic
        count++;
        if (count == nthreads) {
            // Last thread to arrive resets count and flips sense
            count = 0;
            sense = local_sense;
        } else {
            // Other threads wait until sense changes
            while (sense != local_sense) {
                #pragma omp flush(sense)
            }
        }
        // After barrier
        printf("Thread %d passed the barrier\n", tid);
    }
    return 0;
}
```